DOCUMENTAÇÃO FINAL

Clube 21

Blackjack em Flutter
Integração, funcionamento e entrega







Identificação	Versão desta entrega
Aplicativo / documento	Clube 21 0.1.0+1 / Documento 2.0 - 14/10/2026
Tecnologias	Flutter 3.47.2 / Dart 3.13.2 / Deck of Cards API
Pacote Android	c21original.apk - release universal, instalação direta
Código-base	169f17a; lógica e interface validadas no commit 889a701

Evolução da documentação original, versão 1.0 de 21/09/2026, atualizada para a implementação entregue e para os 19 estados do wireframe.

Como usar este documento

Esta edição mantém os 11 assuntos centrais do PDF original e acrescenta os dados do APK, a execução nas seis plataformas e a galeria completa. Os exemplos de implementação correspondem ao código entregue; recursos genéricos da API estão identificados como complementares.

1. Visão geral

2. Contrato HTTP e convenções

3. Catálogo de endpoints

4. Criação, compra e renovação

5. Pilhas e mãos locais

6. Dicionário dos dados

7. Códigos de cartas e regras

8. Estrutura Dart e arquitetura

9. Cliente HTTP e falhas

10. Fluxo e 19 estados

11. Testes e evidências

12. APK e instalação

13. Execução nas seis plataformas

14. Galeria da aplicação

15. Referências e manutenção

Principais atualizações

As classes sugeridas em inglês foram substituídas pelos nomes reais em português. As mãos ficam locais, sem pilhas remotas. O corte cria um novo shoe; compras incertas encerram a mesa. O documento registra testes, builds, limites de validação, instalação e integridade do pacote entregue.

> **Fontes de verdade**
> O comportamento vem de lib/ e test/. As versões resolvidas vêm de pubspec.lock. Os dois PDFs originais permanecem em docs/referencias/. As capturas usam os widgets reais com dados fixos de teste.

1 Visão geral

O Clube 21 é um aplicativo educativo de Blackjack, sem apostas. O jogador enfrenta uma banca automática. A Deck of Cards API fornece o baralho e as cartas; o aplicativo calcula os pontos, resolve as rodadas e salva o estado localmente.

Parte	Comportamento entregue
Mesa	1 a 6 baralhos; padrão 6, totalizando 312 cartas.
Jogador	Pedir carta, parar, iniciar nova rodada e retomar a mesa salva.
Banca	Para no 17 suave por padrão; a preferência permite comprar nessa mão.
Resultados	Blackjack natural, vitória, empate, derrota e estouro.
Preferências	Tema claro/escuro, som, vibração e regras da próxima mesa.
Persistência	Mãos, ID, contagem, regras, fase e última compra no dispositivo.
Interface	Português do Brasil; 19 estados, mensagens de erro e ações bloqueadas durante operações.
Destinos	Web, Android, iOS, Windows, macOS e Linux, com requisitos próprios.

Fora do escopo

Não há apostas, saldo, seguro, divisão de mãos, dobrar, multiplayer, login ou sincronização entre aparelhos. HIT, STAND, soft 17 e shoe são termos do jogo acompanhados de explicações em português.

Dependências diretas resolvidas

Pacote	Versão	Responsabilidade
http	1.6.0	Requisições HTTPS.
shared_preferences	2.5.6	Persistência com SharedPreferencesAsync.
audioplayers	6.8.1	Som opcional.
flutter_lints	6.0.0	Análise estática.

Flutter, flutter_localizations e flutter_test acompanham o SDK. pubspec.yaml contém as restrições; pubspec.lock registra as versões efetivas.

2 Contrato HTTP e convenções

A integração usa a base HTTPS https://deckofcardsapi.com/api/deck/. O aplicativo não precisa de autenticação, token ou servidor próprio. A API embaralha e entrega cartas; a pontuação e o turno da banca pertencem ao Flutter.

Convenção	Aplicação no projeto
Método	GET nos dois endpoints usados. Parâmetros enviados por Uri.https, sem concatenar entradas na query.
Limite de espera	10 segundos por requisição; sem repetição automática de compras.
Resposta válida	HTTP 2xx, JSON objeto, success igual a true e deck_id textual não vazio.
Identificador	O deck_id recebido na criação é guardado na Mesa e conferido em toda compra.
Contagem	remaining inteiro não negativo; aceita número integral ou texto numérico. Ausência e valores fracionários são erros.
Expiração	A API informa remoção após duas semanas de inatividade. O app descarta a mesa salva após 14 dias sem compra.
Mutação via GET	Comprar ou embaralhar altera estado no serviço. Uma falha de rede não prova que a operação deixou de acontecer.

Parâmetros do catálogo original

Nome	Significado
deck_count	Quantidade de baralhos. No app: inteiro de 1 a 6.
count	Quantidade de cartas: 4 para a distribuição e 1 para cada compra seguinte.
cards	Códigos separados por vírgula, em baralho parcial ou operações de pilha.
remaining=true	Embaralha somente as cartas ainda no baralho; não é usado no jogo.
jokers_enabled	Habilita curingas em chamadas compatíveis; não é usado no Blackjack.
pile_name	Nome de uma pilha remota no caminho; não é um estado do aplicativo.

Limite da integração
O app solicita uma operação por vez e valida a resposta inteira antes de adicionar cartas à mão. O campo success não substitui a verificação de contagem, identidade e conteúdo das cartas.

3 Catálogo de endpoints

O wireframe exige apenas criação e compra. Os demais endpoints do PDF original são preservados no catálogo para consulta, mas não fazem parte da execução do Clube 21. Nos caminhos abaixo, {id} representa o deck_id retornado pelo serviço.

Endpoint usado	Parâmetros	Quando acontece
new/shuffle/	deck_count=N	Novo jogo, reinício após compra incerta e renovação do shoe.
{id}/draw/	count=4	Distribuição inicial de cada rodada.
{id}/draw/	count=1	Pedir carta ou compra automática da banca.

Exemplos de URLs

```
https://deckofcardsapi.com/api/deck/new/shuffle/?deck_count=6
https://deckofcardsapi.com/api/deck/ID_RECEBIDO/draw/?count=4
https://deckofcardsapi.com/api/deck/ID_RECEBIDO/draw/?count=1
```

O texto ID_RECEBIDO é apenas um marcador. O programa usa o identificador retornado na criação; ele nunca deve ser fixado no código.

Conferência da criação

Além das convenções HTTP, criar() exige shuffled igual a true e remaining igual a 52 vezes a quantidade escolhida. Para seis baralhos, a resposta precisa informar 312 cartas.

Conferência da compra

comprar() exige o mesmo deck_id, lista com exatamente count cartas e remaining igual à contagem anterior menos count. Cada code deve representar uma carta válida e concordar com value e suit. Nenhuma carta de uma resposta parcial é aplicada.

> **Por que não repetir uma compra**
> Se o serviço consumiu a carta e a resposta se perdeu, reenviar a mesma URL pode comprar outra carta. O estado local já não é confiável; a recuperação oferecida é reiniciar a mesa.

3.1 Endpoints complementares

Referência dos 15 endpoints descritos no PDF original. A base dos caminhos relativos é https://deckofcardsapi.com/api/deck/. Pilhas remotas não são usadas, inclusive pela incompatibilidade documentada com múltiplos baralhos.

Caminho	Função / parâmetros
new/shuffle/?deck_count=N	Cria e embaralha N baralhos. USADO.
new/	Cria baralho ordenado.
new/shuffle/?cards=...	Cria baralho parcial com os códigos indicados.
{id}/draw/?count=N	Compra N cartas. USADO.
{id}/shuffle/	Reembaralha o shoe.
{id}/shuffle/?remaining=true	Embaralha somente as cartas restantes.
{id}/pile/{nome}/add/?cards=...	Adiciona à pilha cartas já retiradas.
{id}/pile/{nome}/shuffle/	Embaralha a pilha.
{id}/pile/{nome}/list/	Lista as cartas da pilha.
{id}/pile/{nome}/draw/	Retira do topo por count ou por cards.
{id}/pile/{nome}/draw/bottom/	Retira cartas da base; count.
{id}/pile/{nome}/draw/random/	Retira cartas aleatórias; count.
{id}/return/	Devolve cartas elegíveis ao baralho; cards é opcional.
{id}/pile/{nome}/return/	Devolve cartas da pilha; cards é opcional.
/static/img/back.png	Verso padrão no domínio da API; fora da base /api/deck/. O app usa verso local.

A forma new/draw/?count=N também é documentada pelo serviço para criar e comprar em uma chamada. Ela não é usada: o app cria explicitamente o shoe com deck_count e só depois compra quatro cartas. A criação parcial do material original usa new/shuffle/?cards=AS,0D,...; não há uma terceira rota de aplicação.

4 Criação, compra e renovação

Novo jogo, em cinco passos

1. Ler a quantidade de baralhos e a regra do 17 suave das preferências.

2. Registrar localmente que há uma operação pendente.

3. Criar o shoe com new/shuffle e conferir o total de cartas.

4. Comprar quatro cartas em uma única chamada draw.

5. Distribuir jogador [0, 2] e banca [1, 3], verificar naturais e salvar o resultado.

A segunda carta da banca fica oculta enquanto o jogador decide. A compra inicial já retirou as quatro cartas do shoe; a interface não faz outra chamada para revelar a carta.

Rodadas seguintes

Após o resultado, Nova rodada limpa as mãos, incrementa o número da rodada e distribui quatro cartas. Enquanto o shoe permanece acima do corte, o deck_id e as regras da mesa são preservados. Alterações nas preferências de baralhos e S17 passam a valer quando uma nova mesa é criada.

Corte do shoe

Ao chegar a 25% ou menos das cartas originais, a próxima rodada exige renovação. A condição também protege o caso de haver menos de quatro cartas. Com seis baralhos, o limite é 78 de 312 cartas. O aviso do corte é mostrado durante a renovação; o aplicativo chama new/shuffle e recebe outro deck_id.

```
bool get renovar =>
    restantes <= baralhos * 52 / 4 || restantes < 4;
```

> **Atualização em relação ao material inicial**
> Renovar significa criar outro shoe, usando as preferências atuais. O aplicativo não chama {id}/shuffle, não devolve cartas e não reaproveita pilhas do shoe anterior.

Concorrência e retomada

Os comandos ficam bloqueados enquanto ocupado é verdadeiro. A mão só recebe cartas depois da validação de toda a resposta. Se o app fechar durante uma operação, a marca pendente impede restaurar uma mesa potencialmente diferente do servidor.

5 Pilhas remotas e mãos locais

O PDF original descreve operações de pile e alerta que elas não funcionam com múltiplos baralhos. A implementação resolve essa limitação mantendo as duas mãos em List<Carta> locais. Isso atende a configuração padrão de seis baralhos sem depender de endpoints de pilha.

Local do dado	O que representa
API: remaining	Cartas que ainda podem ser compradas no shoe.
Mesa.jogador	Cartas já entregues ao jogador na rodada atual.
Mesa.banca	Cartas já entregues à banca, incluindo a carta inicialmente oculta.
API: piles	Coleções remotas do catálogo complementar; nunca criadas pelo aplicativo.

O que é persistido

SharedPreferencesAsync armazena um único texto JSON na chave clube21. O objeto contém versao, preferencias, mesa e pendente. A mesa pode ser nula quando não existe uma partida retomável. Cada carta salva tem codigo e imagem.

```
clube21
  versao: 1
  preferencias: { baralhos, paraNo17Suave, som, vibracao, escuro }
  mesa: { identificador, baralhos, paraNo17Suave, restantes,
          rodada, jogador, banca, fase, ultimaJogada } ou null
  pendente: true ou false
```

Ordem das gravações

Antes de consumir cartas, o controlador tenta salvar pendente=true. Se essa gravação falhar, não consulta a API. Após a ação, salva a mesa confirmada e pendente=false. Uma falha durante a operação invalida a mesa; não há tentativa de recriar localmente uma carta que possa ter sido consumida.

Privacidade e alcance

A partida fica no próprio dispositivo. Na web, o armazenamento depende do navegador e da origem acessada. Não existe conta, nuvem ou transferência automática para outro aparelho. Limpar os dados do aplicativo ou do site remove preferências e mesa salva.

6 Dicionário dos dados

Campo da API	Tipo / significado	Tratamento no app
success	booleano; sucesso da operação	Obrigatoriamente true.
deck_id	texto; identidade do shoe	Não vazio; igual ao da mesa na compra.
shuffled	booleano; baralho embaralhado	Obrigatoriamente true na criação.
remaining	inteiro; cartas restantes	Validado contra o total ou a compra anterior.
cards	lista de objetos carta	Exatamente count itens na compra.
code	texto de dois caracteres	Validado por Carta.ler.
value	ACE, JACK, QUEEN, KING ou número	Precisa concordar com code.
suit	SPADES, HEARTS, DIAMONDS ou CLUBS	Precisa concordar com code.
image / images.png	URLs da frente da carta	Prefere images.png; alternativa image.
images.svg	URL da frente em SVG	Não utilizada.
piles	mapa de pilhas remotas	Não utilizado.

Exemplo didático de resposta de criação

```
{
  "success": true,
  "deck_id": "ID_RECEBIDO",
  "remaining": 312,
  "shuffled": true
}
```

O identificador é ilustrativo. Não use esse valor para executar uma compra. Na compra de uma carta, cards contém o objeto da carta e remaining diminui exatamente em uma unidade.

Contrato público e nomes em português

Os campos JSON e as APIs de bibliotecas mantêm os nomes originais exigidos pelo serviço. No domínio, deck_id vira identificador; remaining vira restantes; code vira codigo; image vira imagem. Essa tradução fica na fronteira HTTP e no modelo, evitando espalhar nomes externos pelas telas.

7 Códigos de cartas e regras

Elemento	Código	Leitura no aplicativo
Ás	A	11 pontos; vira 1 quando necessário.
Números	2 a 9; 0 para dez	O primeiro caractere 0 significa 10.
Figuras	J, Q, K	Valete, dama e rei; 10 pontos.
Naipes	S, H, D, C	Espadas, copas, ouros e paus.

Exemplos: AS é Ás de espadas, 0D é dez de ouros, QH é dama de copas. O formato aceito é ^[A2-90JQK][SHDC]$. Curingas e códigos fora desse conjunto são rejeitados.

Pontuação, diretamente de lib/regras.dart

```
({int pontos, bool suave}) pontuar(List<Carta> cartas) {
  var total = 0;
  var ases = 0;
  for (final carta in cartas) {
    total += carta.pontos;
    if (carta.valor == 'A') ases++;
  }
  while (total > 21 && ases > 0) {
    total -= 10;
    ases--;
  }
  return (pontos: total, suave: ases > 0);
}
```

O cálculo começa com todos os ases em 11. Cada redução subtrai 10, transformando um Ás em 1. suave informa se ainda existe um Ás em 11. Assim, A + 6 é 17 suave; 9 + 7 + A é 17 com o Ás em 1.

Natural e banca

Blackjack natural exige duas cartas e 21 pontos. A banca compra abaixo de 17; no 17 suave, a compra depende da configuração S17/H17. Se o jogador atinge 21 após pedir carta, a vez passa à banca. Se passa de 21, a rodada termina imediatamente.

7.1 Resolução de uma rodada

A ordem importa: um natural vence um 21 formado por três ou mais cartas. O resultado é calculado localmente, sem endpoint adicional.

Ordem	Condição	Resultado
1	Jogador acima de 21	Você passou de 21; banca vence.
2	Ambos com natural	Empate.
3	Somente banca com natural	Blackjack da banca.
4	Somente jogador com natural	Blackjack!
5	Banca acima de 21	Você venceu.
6	Pontos iguais	Empate.
7	Pontos diferentes, sem estouro	Vence quem tem mais pontos.

Exemplos para explicar o código

Mão	Pontos	Interpretação
A + A + 9	21 suave	Um Ás vira 1; o outro permanece 11. Não é natural.
9 + 7 + A	17	O Ás vira 1, evitando 27.
A + K	21	Duas cartas: Blackjack natural.
10 + 7	17	Mão sem Ás em 11; a banca para nas duas configurações.
A + 6	17 suave	S17: para. H17: compra.

Preferências da mesa

Os valores de baralhos e paraNo17Suave são copiados ao criar a Mesa e permanecem fixos nela. Isso impede uma mudança nas configurações de alterar a regra de uma rodada já iniciada. Tema, som e vibração são preferências de apresentação.

8 Estrutura Dart e arquitetura

O projeto concentra o aplicativo em cinco arquivos. Cada um tem uma responsabilidade clara; comentários explicam as regras e os pontos em que a execução precisa de cuidado. O domínio usa nomes em português, preservando os nomes obrigatórios de Flutter, Dart e da API.

Arquivo	Responsabilidade	Leitura recomendada
lib/regras.dart	Carta, Preferencias, Mesa, pontuação e resultado.	Comece aqui: não depende de internet ou widgets.
lib/api.dart	Criação do shoe, compra e validação das respostas.	Mostra exatamente as URLs e campos externos.
lib/jogo.dart	Turnos, operações assíncronas, salvamento e recuperação.	Coordena dados e regras; notifica mudanças.
lib/telas.dart	Menu, mesa, configurações, regras e estados visuais.	Transforma o estado do Jogo em interface.
lib/main.dart	Inicialização, tema, localização e ciclo de vida.	Conecta o controlador ao aplicativo.

Caminho de uma ação

```
Toque em Pedir carta
  -> tela chama Jogo.pedirCarta()
  -> controlador grava pendente=true
  -> ApiBaralho.comprar(mesa, 1) consulta a API
  -> resposta inteira validada
  -> Mesa recebe a carta e a nova contagem
  -> pontuar() decide se continua, passa a vez ou estoura
  -> controlador salva o estado e notifica a tela
```

Por que as dependências são injetadas

ApiBaralho recebe um http.Client; Jogo recebe a API e um Armazenamento. Os testes substituem esses dois pontos por implementações controladas. Isso permite simular falhas, contar requisições e verificar salvamentos sem consumir o baralho público.

8.1 Modelos e validação local

Nome real	Conteúdo
Carta	codigo, imagem; getters valor, simbolo, vermelha, pontos e descricao.
Preferencias	baralhos, paraNo17Suave, som, vibracao e escuro.
Mesa	identificador, regras fixas, restantes, rodada, mãos, fase e ultimaJogada.
Fase	jogador, banca e concluida.
Jogo	Estado observável: mesa, preferencias, ocupado, erro, aviso e indicadores de recuperação.
Armazenamento	Contrato mínimo: ler() e salvar(conteudo).
MemoriaLocal	Implementação com SharedPreferencesAsync e a chave clube21.

Os DTOs PlayingCard, DeckResponse, DrawResponse, PileResponse e o GameState sugeridos no PDF inicial não foram criados como camadas extras. O cliente converte a resposta diretamente para Mesa e para um registro com cartas e restantes. A estrutura menor preserva as validações necessárias.

Recuperação de uma mesa salva

Mesa.ler rejeita identificador vazio, quantidade de baralhos fora de 1 a 6, contagem negativa ou maior que o total, rodada menor que 1, mãos com menos de duas cartas e a fase banca. Tipos, códigos, data e fase inválidos também impedem recuperar o objeto.

Jogo.carregar valida a versão do formato, verifica a marca pendente e a idade da sessão. Uma mesa inválida ou expirada é descartada com aviso; o aplicativo ainda abre o menu. A fase banca não é retomada pela metade porque suas compras podem ter sido interrompidas.

Valores padrão das preferências

Seis baralhos, banca para no 17 suave, som ligado, vibração ligada e tema escuro. Ao ler preferências válidas, baralhos é limitado ao intervalo de 1 a 6. Se uma alteração não puder ser gravada, o controlador restaura a configuração anterior e informa o problema.

9 Cliente HTTP

Este método de lib/api.dart concentra URL, prazo de espera e validações comuns. O cliente usa uma única tentativa; criar() e comprar() complementam as verificações de acordo com a operação.

```
Future<Map<String, dynamic>> _consultar(
  String caminho,
  Map<String, String> parametros,
) async {
  final resposta = await cliente
      .get(Uri.https('deckofcardsapi.com', '/api/deck/$caminho/', parametros))
      .timeout(limite);
  if (resposta.statusCode < 200 || resposta.statusCode >= 300) {
    throw const FormatException(
      'O serviço de cartas não respondeu corretamente.',
    );
  }
  final dados = jsonDecode(resposta.body);
  if (dados is! Map<String, dynamic> ||
      dados['success'] != true ||
      dados['deck_id'] is! String ||
      (dados['deck_id'] as String).isEmpty) {
    throw const FormatException('Resposta inválida do serviço de cartas.');
  }
  return dados;
}
```

Imagem das cartas

A frente de uma carta aberta tenta carregar images.png, usando image como alternativa. Se a imagem estiver ausente ou falhar, a própria interface desenha valor e naipe. Isso mantém a mão legível sem tornar o SVG uma dependência.

A carta fechada usa verso local. Sua frente não é carregada nem exposta por texto de acessibilidade; o leitor recebe a descrição Carta fechada. Ao revelar, o app usa a carta já recebida na distribuição, sem comprar outra.

9.1 Falhas e recuperação

Situação	Resposta do aplicativo	Ação do usuário
Falha ao criar shoe	Mesa não fica disponível; mostra erro de conexão.	Tentar novamente cria outro shoe.
Timeout ou resposta inválida ao comprar	Compra incerta; mesa invalidada.	Reiniciar a mesa com outro baralho.
Falha ao gravar antes da operação	Não envia a requisição remota.	Corrigir o armazenamento e tentar novamente.
Falha ao salvar depois da jogada	Encerra a mesa para evitar retomada antiga.	Corrigir o armazenamento e iniciar nova mesa.
Fechamento com operação pendente	Recusa retomar a sessão na próxima abertura.	Iniciar nova mesa.
JSON local inválido ou sessão expirada	Abre o menu com aviso e sem mesa recuperável.	Novo jogo.
Imagem da carta falha	Desenha valor e naipe localmente.	Continuar jogando.
Som ou vibração indisponível	Efeito opcional é ignorado.	Conferir volume, preferências e suporte.

O que não fazer na manutenção

Não introduzir repetição automática de draw, recuperação por contagem estimada ou aplicação parcial da lista de cartas. Essas alterações podem deixar a mão local diferente do shoe remoto. Não retornar zero silenciosamente quando remaining estiver ausente.

Validação sem internet

Os testes substituem a API e o armazenamento. Eles exercitam respostas malformadas, HTTP inválido, falhas em gravações e ordem das operações. O uso real ainda exige internet para criar o shoe e obter cartas; o app não implementa um modo de jogo totalmente offline.

10 Fluxo da aplicação

```
Abertura -> recuperar preferências e sessão -> Menu

Novo jogo -> criar shoe -> comprar 4 -> verificar naturais
  natural de alguém -> Resultado
  sem natural -> Sua vez

Sua vez -> Pedir -> comprar 1 -> recalcular
  menos de 21 -> Sua vez
  exatamente 21 -> Banca
  acima de 21 -> Estouro

Sua vez -> Parar -> revelar carta local -> Banca
Banca -> comprar 1 enquanto a regra exigir -> Resultado

Resultado -> Nova rodada
  acima do corte -> comprar 4 no mesmo shoe
  no corte -> criar outro shoe -> comprar 4

Pausa -> Continuar jogando OU Salvar e ir ao menu
Menu -> Continuar -> retomar a mão sem nova compra
```

Interface e fidelidade à referência

O app mantém o fundo de mesa verde, os acentos dourados, as cartas sobrepostas, as mensagens de turno, os painéis de resultado e o modal de pausa do wireframe. Os 19 estados foram exportados a partir dos próprios widgets Flutter.

Referência	Adaptação implementada
Prancha 390 × 844	Área segura, conteúdo responsivo, largura limitada e rolagem quando necessária.
Georgia e Segoe UI	Gelasio nos títulos e Roboto no texto; fonte incluída para os naipes.
Relógio e bateria desenhados	Barras reais do sistema, sem relógio ou bateria fictícios.
Cartas ilustrativas fixas	Cartas da API no jogo; dados controlados na galeria.
Tema escuro	Tema escuro da referência e alternativa clara prevista nas configurações.

A conferência é funcional e visual, com as adaptações declaradas acima. Não se apresenta uma reprodução pixel a pixel de fontes, barras do sistema ou cartas aleatórias.

10.1 Estados 01 a 10

Nº	Tela	Comportamento e chamadas
01	Abertura	Lê dados locais e mostra preparação. Nenhuma chamada de cartas.
02	Menu sem partida	Continuar desabilitado. Novo jogo inicia new/shuffle.
03	Menu com partida	Retoma dados locais. Substituir a mesa exige confirmação na interface.
04	Configurações	1 a 6 baralhos, S17/H17, efeitos e tema. Grava localmente.
05	Como jogar	Explica objetivo, pontos e ações. Sem HTTP de cartas.
06	Distribuição	Confirma criação e compra count=4. Bloqueia ações.
07	Sua vez	Pedir usa count=1; Parar revela a carta local.
08	Ás flexível	Exibe 9 + 7 + A = 17 e explica o Ás valendo 1.
09	Vez da banca	Revela e compra count=1 por vez quando necessário.
10	Blackjack do jogador	Resolve natural nas duas cartas iniciais sem compra extra.

Ações locais

Menu, regras, configurações, pausa, retomada, revelação da carta e cálculo do resultado não exigem endpoints adicionais. A rede só fornece o shoe, compras e imagens de cartas abertas.

Acessibilidade e espaço disponível

A carta oculta não revela valor ou naipe na árvore semântica. A interface também foi testada com largura de 320 pixels e texto a 160%. A rolagem permite acessar conteúdos maiores sem exigir a resolução fixa da prancha.

10.2 Estados 11 a 19

Nº	Tela	Comportamento e chamadas
11	Vitória	Vitória por pontos ou estouro da banca. Nova rodada disponível.
12	Empate	Pontuação igual ou dois naturais. Cálculo local.
13	Derrota	Painel de derrota por pontos; permite outra rodada.
14	Estouro do jogador	Final imediato, sem compras adicionais da banca.
15	Erro de conexão	Falha na criação. Tentar novamente cria outro shoe.
16	Compra incerta	Mesa invalidada. Reiniciar usa criação e nova distribuição.
17	Renovação do shoe	Entre rodadas, quando restantes ≤ 25% ou abaixo de quatro.
18	Pausa	Continuar jogando ou Salvar e ir ao menu. Sem compra.
19	Blackjack da banca	Resolvido na distribuição; empate se ambos forem naturais.

Como reproduzir as capturas

As imagens são geradas por test/wireframe_test.dart com os componentes reais, dados fixos e resolução de 780 × 1688 (2× a prancha). A galeria usa a frente local das cartas para ser determinística.

```
# PowerShell: ajuste para sua instalação do Flutter.
$env:FLUTTER_ROOT = 'C:\flutter'
flutter test test/wireframe_test.dart --dart-define=EXPORTAR_TELAS=true

# macOS / Linux: ajuste para sua instalação.
export FLUTTER_ROOT="$HOME/development/flutter"
flutter test test/wireframe_test.dart --dart-define=EXPORTAR_TELAS=true
```

As capturas ficam em docs/imagens/. Elas comprovam a renderização dos estados controlados, e não representam fotografias ou testes de um telefone físico.

11 Testes e evidências

Verificação	Resultado registrado
Formatação / análise	dart format sem alterações; flutter analyze sem problemas.
Testes automatizados	47 testes aprovados: regras, cliente HTTP, controlador, persistência e interface.
19 estados do wireframe	Renderizados em teste e exportados para docs/imagens/.
Web	Build release local e no GitHub; rodada com API real, pausa e retomada após recarga verificadas no navegador.
Android	APK debug local e no GitHub; APK release final gerado localmente nesta entrega.
Windows	Build release aprovado no GitHub. Visual Studio 2019 local falhou no plugin de áudio; usar compilador atual.
Linux e macOS	Builds release aprovados no GitHub.
iOS	Build de simulador em debug aprovado no GitHub, sem assinatura de distribuição.
APK final	Manifesto e assinatura verificados; SHA-256 registrado no capítulo 12.

Execução rastreável

GitHub Actions - execução 38658824175 compilou os seis destinos para o código do commit 889a701. O commit 169f17a acrescentou o registro dos resultados. O código do aplicativo permaneceu o mesmo para o APK release desta entrega.

Reexecutar os controles

```
dart format --output=none --set-exit-if-changed lib test
flutter analyze
flutter test
```

> **Limites da validação**
> Compilação aprovada não substitui execução em hardware. Não foi feita instalação deste APK em telefone físico; iOS, macOS, Linux e Windows também precisam de teste de uso no dispositivo de apresentação. A disponibilidade da API e os efeitos dependem da rede e da plataforma.

12 APK final e instalação

Propriedade	Valor conferido no artefato
Arquivo	c21original.apk, na raiz do repositório.
Versão	0.1.0; versionCode 1 (0.1.0+1).
Nome / pacote	Clube 21 / br.com.clube21.clube_21
Modo	Release universal: armeabi-v7a, arm64-v8a e x86_64.
Tamanho	50.312.103 bytes; aproximadamente 48,0 MiB.
Sistema mínimo	Android 7.0 (API 24).
SDK alvo / compilação	API 36 / API 36, conforme o manifesto.
Assinatura	Certificado Android Debug de desenvolvimento; verificação criptográfica aprovada.
Permissões	Internet e vibração; permissão interna do pacote para receptores não exportados.

Integridade SHA-256

```
6C14B70FF794E24E63EFFD3C721AA01B
A2E1F06F91103C01C0EE96087F371C5F
```

O hash está dividido em duas linhas apenas para caber na página. Una os 64 caracteres para comparar. Se o APK for recompilado, o hash pode mudar; atualize este registro.

Instalar no celular ou tablet Android

1. Baixe ou copie c21original.apk para o aparelho com Android 7.0 ou superior.
2. Abra o arquivo pelo navegador ou gerenciador de arquivos.
3. Quando o Android solicitar, autorize essa origem a instalar aplicativos.
4. Conclua a instalação e abra Clube 21.
5. Com internet, toque em Novo jogo; teste Pedir carta, Parar e a retomada pelo menu.

O pacote usa otimizações de release, mas a assinatura atual é de desenvolvimento para demonstração. Publicação em loja exige chave própria e configuração de distribuição. Não há certificados privados nem senhas no repositório.

12.1 Instalar por USB ou recompilar

Instalação pelo computador

Instale Android Platform Tools, habilite a depuração USB no Android, conecte o cabo e aceite a autorização. Depois execute na raiz do projeto:

```
adb devices
adb install -r c21original.apk
```

Se o Android informar conflito de assinatura, a versão instalada foi assinada com outra chave. Para substituí-la, será necessário removê-la primeiro; isso apaga sua partida local. Se o modelo não aceitar o pacote, confira a versão do Android e a mensagem exibida pelo sistema.

Conferir o hash no Windows

```
Get-FileHash -Algorithm SHA256 .\c21original.apk
```

Conferir no macOS ou Linux

```
# macOS
shasum -a 256 c21original.apk
# Linux
sha256sum c21original.apk
```

Gerar novamente a partir do código

```
flutter pub get
flutter build apk --release
```

A saída é build/app/outputs/flutter-apk/app-release.apk. Para atualizar o arquivo da raiz no PowerShell:

```
Copy-Item build/app/outputs/flutter-apk/app-release.apk c21original.apk
```

No macOS ou Linux:

```
cp build/app/outputs/flutter-apk/app-release.apk c21original.apk
```

> **Distribuição por plataforma**
> O APK instala apenas no Android. iPhone e iPad usam a versão web ou uma compilação iOS preparada no Mac. Windows, Linux e macOS usam a web ou seus próprios builds desktop.

13 Preparar o ambiente

Todos os comandos deste capítulo partem da raiz do projeto. Você só precisa dos requisitos da plataforma desejada. Para instalar o APK pronto, não é necessário instalar Flutter no telefone.

1. Instale Git e o Flutter SDK 3.47.2, que inclui Dart 3.13.2.

2. Adicione flutter/bin ao PATH e reabra o terminal.

3. Execute flutter --version e flutter doctor -v.

4. Corrija os itens da plataforma que pretende executar.

5. Clone o repositório e baixe as dependências:

```
git clone https://github.com/bernardohorn/blackjack.git
cd blackjack
flutter pub get
flutter devices
```

Quando um exemplo usar ID_DO_DISPOSITIVO, substitua pelo identificador que flutter devices mostrar. Não são necessários arquivo .env, conta de usuário, chave de API ou banco de dados.

Requisitos por destino

Destino	Computador de preparação	Ferramentas principais
Web	Windows, macOS ou Linux	Flutter e navegador compatível.
Android	Windows, macOS ou Linux	Android Studio, SDK e Platform Tools.
iOS	macOS	Xcode, simulador ou aparelho e assinatura quando exigida.
Windows	Windows	Visual Studio 2022 ou posterior com C++, MSVC, CMake e Windows SDK.
macOS	macOS	Xcode e componentes de plugins.
Linux	Linux compatível	Clang, CMake, Ninja, GTK e GStreamer.

Os passos completos também estão em docs/EXECUTAR.md. As versões e requisitos dos fornecedores podem mudar; os links oficiais no capítulo 15 ajudam a ajustar a instalação.

13.1 Web: computador e celular

Executar no computador

Instale Chrome e execute:

```
flutter run -d chrome
```

No Windows, Edge também pode ser usado com -d edge se aparecer na lista de dispositivos.

Abrir no celular ou tablet pela mesma rede

1. Conecte o computador e o aparelho à mesma rede Wi-Fi.

2. No computador, inicie o servidor:

```
flutter run -d web-server --web-hostname 0.0.0.0 --web-port 8765
```

3. Descubra o IPv4 local: ipconfig no Windows; ipconfig getifaddr en0 no macOS com essa interface Wi-Fi; hostname -I no Linux. Se necessário, consulte os ajustes de rede do sistema.

4. No aparelho, abra http://IP_DO_COMPUTADOR:8765, por exemplo http://192.168.1.10:8765. Mantenha o computador e o terminal ligados. Se o firewall perguntar, permita o servidor na rede privada utilizada.

> **Endereço correto**
> 127.0.0.1 e localhost no telefone apontam para o próprio telefone. Para acessar o computador, use o IPv4 dele. Redes de visitantes podem impedir a comunicação entre aparelhos.

Compilar e pré-visualizar a versão de entrega

```
flutter build web --release
python -m http.server 8765 --directory build/web
```

O segundo comando exige Python 3. Abra http://localhost:8765; não abra index.html diretamente pelo gerenciador de arquivos. Para hospedagem, envie todo o conteúdo de build/web/ a um servidor estático com HTTPS.

Hospedagem em uma subpasta

```
flutter build web --release --base-href /blackjack/
```

No celular, o suporte a som, vibração e atalho na tela inicial varia entre navegadores. As compras de cartas exigem internet.

13.2 Android: executar e compilar

1. Instale Android Studio, Android SDK, Command-line Tools e Platform Tools. Para emulação, instale Android Emulator e crie um aparelho no Device Manager.

2. Execute flutter doctor --android-licenses e aceite as licenças dos componentes utilizados.

3. Confira flutter doctor -v. O projeto usa o JDK do Android Studio e alvo Java 17.

4. Inicie um emulador ou conecte um aparelho com depuração USB habilitada. No Windows, o fabricante pode exigir driver USB.

5. Aceite a autorização no aparelho e execute:

```
flutter devices
flutter run -d ID_DO_DISPOSITIVO
```

Build para desenvolvimento

```
flutter build apk --debug
```

Saída: build/app/outputs/flutter-apk/app-debug.apk. Esse arquivo é diferente do release otimizado entregue na raiz.

Build de entrega

```
flutter build apk --release
```

Saída: build/app/outputs/flutter-apk/app-release.apk. A configuração de assinatura atual usa o certificado de desenvolvimento também nesse modo; consulte o capítulo 12 antes de distribuir.

Problemas frequentes

Sintoma	Conferir
Nenhum aparelho listado	Cabo de dados, depuração USB, autorização, driver e adb devices.
Erro de SDK ou licenças	flutter doctor -v e flutter doctor --android-licenses.
Falha ao instalar	Android mínimo, espaço livre e conflito de assinatura com outra instalação.
Erro ao criar a mesa	Conexão com internet e disponibilidade da API.

13.3 iPhone e iPad

A compilação nativa requer um Mac. Em Windows ou Linux, o iPhone/iPad pode acessar a versão web pelo navegador, seguindo o capítulo 13.1.

Simulador

1. Instale Xcode compatível com seu ambiente Flutter e conclua a configuração inicial, incluindo a plataforma iOS.

2. Configure as ferramentas de linha de comando, aceite a licença do Xcode e execute flutter doctor -v. Instale os componentes indicados para os plugins.

3. Execute flutter pub get e abra um simulador de iPhone ou iPad pelo Xcode.

4. Identifique o simulador e execute:

```
flutter devices
flutter run -d ID_DO_SIMULADOR
```

iPhone ou iPad físico

1. Conecte o aparelho ao Mac e confirme que confia no computador. Habilite o Modo de Desenvolvedor quando solicitado.

2. Abra ios/Runner.xcworkspace no Xcode.

3. Em Runner > Signing & Capabilities, escolha seu Team e um Bundle Identifier disponível para sua conta.

4. Selecione o dispositivo e execute pelo Xcode ou pelo Flutter:

```
flutter run -d ID_DO_DISPOSITIVO
```

Compilação sem assinatura para simulador

```
flutter build ios --simulator --debug
```

Distribuição com assinatura configurada

```
flutter build ipa --release
```

A distribuição exige os certificados e as permissões correspondentes da conta Apple. O repositório não inclui certificados. A evidência automatizada desta entrega é o build de simulador em debug, não um IPA publicado.

13.4 Windows e macOS

Windows

1. Em um Windows, instale Visual Studio 2022 ou posterior com Desenvolvimento para desktop com C++, MSVC, CMake e Windows SDK. Visual Studio Code sozinho não fornece esse compilador.

2. Execute flutter doctor -v e corrija a seção Windows.

3. Baixe dependências e execute:

```
flutter pub get
flutter run -d windows
flutter build windows --release
```

Para x64, a saída fica em build/windows/x64/runner/Release/. Distribua a pasta inteira, incluindo DLLs e data/. A máquina de destino pode precisar do runtime Visual C++. Use a arquitetura correspondente.

> **Compilador validado**
> O build Windows passou no GitHub. A tentativa local com Visual Studio 2019 falhou em corrotinas C++ do plugin de áudio; use a ferramenta atual indicada no guia.

macOS

1. Em um Mac, instale Xcode e conclua a configuração inicial.

2. Execute flutter doctor -v e instale os componentes de plugins solicitados.

3. Execute os comandos:

```
flutter pub get
flutter run -d macos
flutter build macos --release
```

O .app fica em build/macos/Build/Products/Release/. Os entitlements do projeto permitem conexão de saída para consultar a API. Distribuição pública pode exigir assinatura e notarização da Apple.

13.5 Linux e solução de problemas

Em Ubuntu/Debian, instale as ferramentas e bibliotecas a seguir. Em outras distribuições, use os pacotes equivalentes e confira o guia oficial de Flutter para Linux.

```
sudo apt-get update
sudo apt-get install -y clang cmake ninja-build pkg-config \
  libgtk-3-dev liblzma-dev libstdc++-12-dev \
  libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev \
  gstreamer1.0-plugins-base gstreamer1.0-plugins-good
flutter doctor -v
flutter pub get
flutter run -d linux
flutter build linux --release
```

Para x64, distribua todo o diretório build/linux/x64/release/bundle/. A máquina de destino precisa de GTK e das bibliotecas de execução do GStreamer. O caminho varia conforme a arquitetura.

Diagnóstico comum às plataformas

Sintoma	Como resolver
flutter não reconhecido	Adicione flutter/bin ao PATH e reabra o terminal.
Falha de compilação	Use flutter doctor -v e corrija o requisito da plataforma.
Partida não aparece em outro aparelho	O salvamento é local; não há sincronização por conta.
Rodada interrompida	Inicie uma nova mesa; não repita a compra antiga.
Não foi possível salvar	Confira espaço e armazenamento local; evite modos que bloqueiem persistência na web.
Sem áudio ou vibração	Confira preferências, volume e suporte. Os efeitos são opcionais.
Mesa expirada	Crie outro shoe; sessões com 14 dias sem compra são descartadas.

14 Galeria da aplicação

Widgets reais em estados controlados; frente local das cartas. Capturas completas em docs/imagens/.

01 Abertura



02 Menu



03 Menu com partida



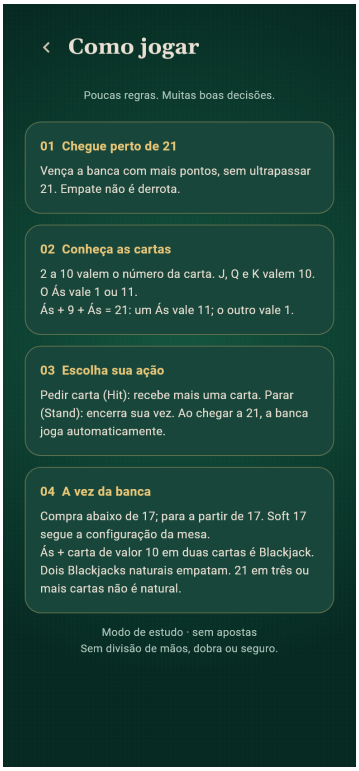
04 Configurações



14 Galeria - continuação

Widgets reais em estados controlados; frente local das cartas. Capturas completas em docs/imagens/.

05 Como jogar



06 Distribuição



07 Sua vez



08 Ás flexível



14 Galeria - continuação

Widgets reais em estados controlados; frente local das cartas. Capturas completas em docs/imagens/.

09 Vez da banca



10 Blackjack



11 Vitória



12 Empate



14 Galeria - continuação

Widgets reais em estados controlados; frente local das cartas. Capturas completas em docs/imagens/.

13 Derrota



14 Estouro



15 Erro de conexão



16 Compra incerta



14 Galeria - continuação

Widgets reais em estados controlados; frente local das cartas. Capturas completas em docs/imagens/.

17 Renovação



18 Pausa



19 Blackjack da banca



15 Referências e manutenção

Material do projeto

Repositório GitHub - Código e histórico da entrega.

PDF técnico original - Versão 1.0, 21/09/2026; base dos capítulos 1 a 11.

Wireframe original - Prancha com os 19 estados.

Guia de execução - Passos completos em Markdown.

Relatório de conformidade - Auditoria, adaptações e evidências.

Guia do código - Leitura dos cinco arquivos do aplicativo.

Galeria - Capturas em resolução integral.

GitHub Actions - Evidência das seis compilações.

Fontes técnicas oficiais

Deck of Cards API: contrato HTTP e recursos

Flutter: arquivo de versões do SDK

Flutter: ambiente Android

Flutter: assinatura e distribuição Android

Flutter: ambiente iOS

Flutter: ambiente Windows

Flutter: ambiente macOS

Flutter: ambiente Linux

Audioplayers: bibliotecas de áudio no Linux

Atualização da entrega

Ao alterar o aplicativo, execute análise e testes, atualize as capturas afetadas e gere outro APK. Confira versão, pacote, SDK mínimo, assinatura e SHA-256; atualize este PDF e o README antes do commit. Não versione caches, builds temporários, chaves de assinatura ou senhas. O APK final na raiz é uma exceção intencional aos arquivos de build.

Créditos: Deck of Cards API para as cartas; Gelasio e Noto Sans Symbols 2 com licenças OFL em assets/fontes/. O som de carta é um recurso local do projeto. Documento preparado para a entrega de 14/10/2026.